

AI FAQ Assistant AI - AI-powered FAQ and customer support automation.

Team ID: SWTID-2026-3407

Team Leader: PRIYANKA A & priyankaofficial5518@gmail.com

Team Member: POOVARASAN & akilaaadhi2623@gmail.com

Team Member: SURIYA R & Gokulsurya267@gmail.com

Team Member: VAISHNAVI & Vaishnavi.goodgirl22@gmail.com

Project Overview

- The **AI FAQ Assistant** is a secure and intelligent backend application developed using **Node.js, Express.js, and Mongo DB**. The main purpose of this project is to help users create, manage, search, and generate Frequently Asked Questions (FAQs) using Artificial Intelligence.
- The system provides **JWT-based authentication** to protect private APIs and uses **bcrypt** to securely encrypt user passwords. Users can create, update, delete, and search FAQ records based on their access permissions.
- The project also integrates **Google Gemini AI** to automatically generate FAQ questions and answers based on a given topic. MongoDB is used to store user and FAQ information, while **Mongoose** is used for schema validation and database operations.
- Overall, the AI FAQ Assistant provides a **secure, organized, and AI-powered platform** for managing FAQ content through RESTful APIs.

Key Objectives:

- **Secure User Authentication** – Implement JWT-based login and authentication to protect private APIs.
- **FAQ Management** – Allow users to create, view, update, and delete FAQ records.
- **AI-Powered FAQ Generation** – Use Google Gemini AI to automatically generate FAQ questions and answers from a given topic.
- **Smart FAQ Search** – Provide keyword-based search to help users quickly find relevant FAQs.
- **Data Security** – Protect user passwords using bcrypt hashing and prevent unauthorized access.
- **Category Management** – Organize FAQs into different categories for easy navigation and management.
- **Data Validation** – Use Mongoose schemas to validate data before storing it in MongoDB.
- **Error Handling** – Provide centralized error handling for reliable and consistent API responses.
- **RESTful API Development** – Build structured and scalable APIs using Node.js and Express.js.

Target Users:

The **AI FAQ Assistant** is mainly designed for the following users:

- **Content Creators** – Can create, edit, delete, and organize FAQ content.
- **Customer Support Teams** – Can quickly search FAQs and use AI-generated answers to handle repeated customer questions.
- **Administrators** – Can manage users, FAQs, categories, and monitor the overall system.
- **Authenticated Users** – Can view FAQs, search for information, and use AI-assisted features.
- **Public Users / Consumers** – Can view published FAQs and search for relevant information without accessing private features.

- **Students and Beginner Developers** – Can use the project as a practical example of Node.js, Express.js, MongoDB, JWT authentication, and AI integration.

Project Instruction

AI FAQ Assistant is a backend-only project built using **Node.js, Express.js, MongoDB, and Mongoose**. It provides secure REST APIs for user authentication, FAQ management, FAQ search, and AI-powered FAQ generation using Google Gemini.

Quick Start

```
npm init
npm install
npm run dev
```

Steps to Create the Project

- Choose or create a directory for the backend project.
- Open the terminal and navigate to that directory.
- Initialize the Node.js project using:

```
npm init
```

- Install the required packages:

```
npm install express mongoose jsonwebtoken bcrypt cors dotenv @google/genai
```

- Create the required folders:
 - models
 - routes
 - controllers
 - middleware
 - config

- Configure MongoDB connection and environment variables.
- Implement user registration and login with JWT authentication.
- Create FAQ APIs for creating, viewing, updating, and deleting FAQs.
- Implement FAQ search functionality.
- Integrate Google Gemini AI for automatic FAQ generation.
- Start the development server using:

Pre-requisites

Software	Purpose
Node.js	JavaScript runtime for backend development
MongoDB	NoSQL database for storing users and FAQs
npm	Package manager for installing project dependencies
Express.js	Framework for creating RESTful APIs
Mongoose	ODM for MongoDB schema and database operations
Postman / Thunder Client	API testing and verification
Visual Studio Code	Code editor for project development
Google Gemini API	AI-powered FAQ generation

Main Technologies

- **Node.js** – Runs the backend application.
- **Express.js** – Handles API routes and server requests.
- **MongoDB** – Stores user and FAQ data.
- **Mongoose** – Defines schemas and validates database data.
- **JWT** – Provides secure user authentication.
- **bcrypt** – Hashes user passwords securely.
- **Google Gemini AI** – Generates FAQ questions and answers.

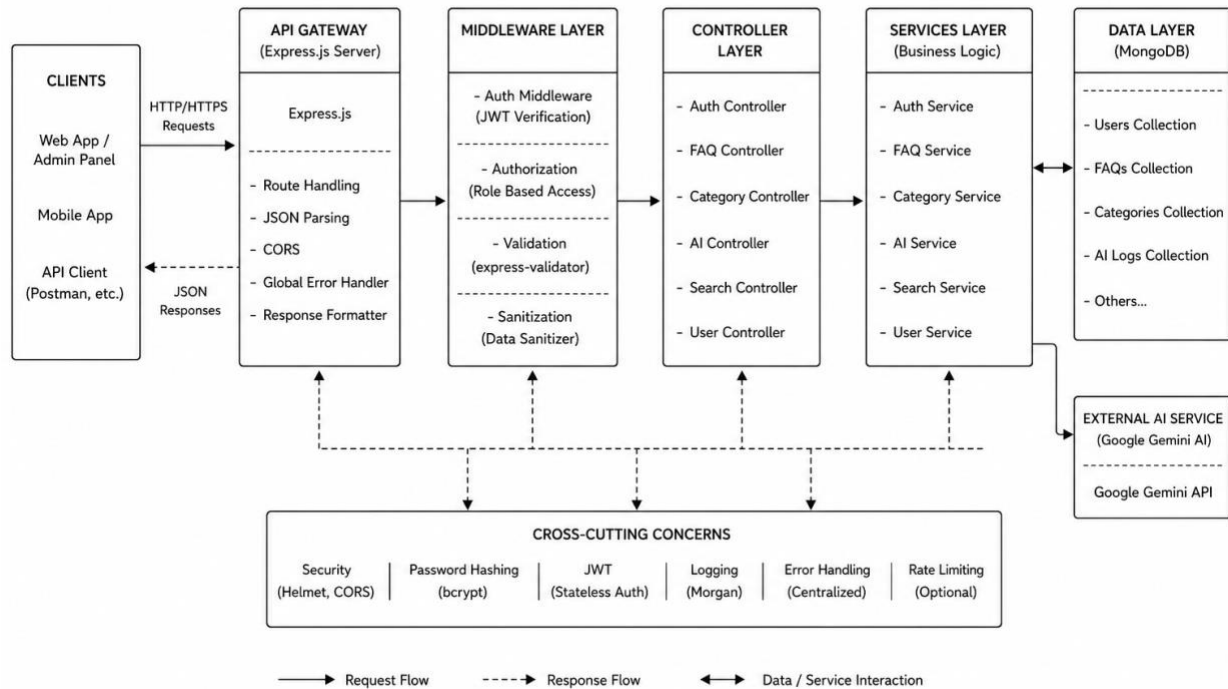
- **Postman / Thunder Client** – Tests the REST APIs.

Knowledge Requirements

- **JavaScript Basics** – Variables, functions, and async programming.
- **Node.js & Express.js** – Backend development and REST APIs.
- **MongoDB & Mongoose** – Database and schema management.
- **JWT & bcrypt** – Authentication and password security.
- **API Testing** – Postman or Thunder Client.
- **AI Integration** – Basic knowledge of Google Gemini API.

Project Architecture Image

AI FAQ Assistant API – Technical Architecture



Technical Architecture

The AI FAQ Assistant follows a **layered backend architecture** using:

- **Client Layer** – Postman / Thunder Client sends API requests.
- **Express.js Server** – Handles HTTP requests and API routes.
- **Authentication Middleware** – Verifies JWT tokens and user access.
- **Controller Layer** – Handles application logic and API responses.
- **Mongoose Layer** – Manages schemas and database operations.
- **MongoDB** – Stores user and FAQ data.

- **Gemini AI Service** – Generates AI-powered FAQ content.

Database Architecture

The AI FAQ Assistant uses **MongoDB** as the primary NoSQL database and **Mongoose** for schema management and validation.

Main Collections

- **User Collection** – Stores user name, email, hashed password, and role.
- **FAQ Collection** – Stores questions, answers, categories, and creator details.
- **Category** – Organizes FAQs based on different topics.
- **AI Generated FAQ** – Stores AI-generated question and answer information when required.

Relationships

- **User → FAQ:** One user can create multiple FAQs.
- **Category → FAQ:** One category can contain multiple FAQs.
- **User → AI FAQ:** One user can generate multiple AI-assisted FAQs.

Database Flow:

Node.js → Mongoose → MongoDB → Users / FAQs / Categories

Features:

- **JWT Authentication** – Provides secure user login and protected API access.
- **AI FAQ Generation** – Uses Google Gemini AI to generate FAQ questions and answers.
- **FAQ CRUD Operations** – Allows users to create, view, update, and delete FAQs.
- **FAQ Search** – Helps users quickly find FAQs using keywords.
- **Category Management** – Organizes FAQs into different categories.
- **Password Security** – Protects passwords using bcrypt hashing.

- **Data Validation** – Mongoose validates data before storing it in MongoDB.
- **Error Handling** – Provides consistent and reliable API error responses.
- **Role-Based Access** – Controls features based on user roles and permissions.

Roles and Responsibilities:

Student / Developer:

- Write backend code using Node.js and Express.js
- Create RESTful APIs
- Integrate MongoDB and Gemini AI
- Test APIs using Postman / Thunder Client
- Implement authentication and security
- Push code to GitHub

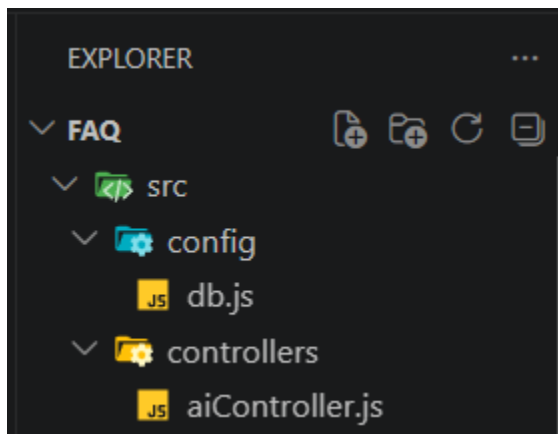
System (Backend):

- Authenticate users using JWT
- Handle FAQ CRUD operations
- Store and fetch data from MongoDB
- Protect private API routes
- Generate FAQs using Gemini AI
- Validate data using Mongoose
- Handle API errors centrally

PROJECT SETUP AND CONFIGURATION

Creating Project Folder

1. Create a new folder with your <project name>.
2. Inside that folder, create a new subfolder named src.
3. Now open that folder in VS Code



Server Setup (npm init)

Open your root folder terminal in VS Code and run:

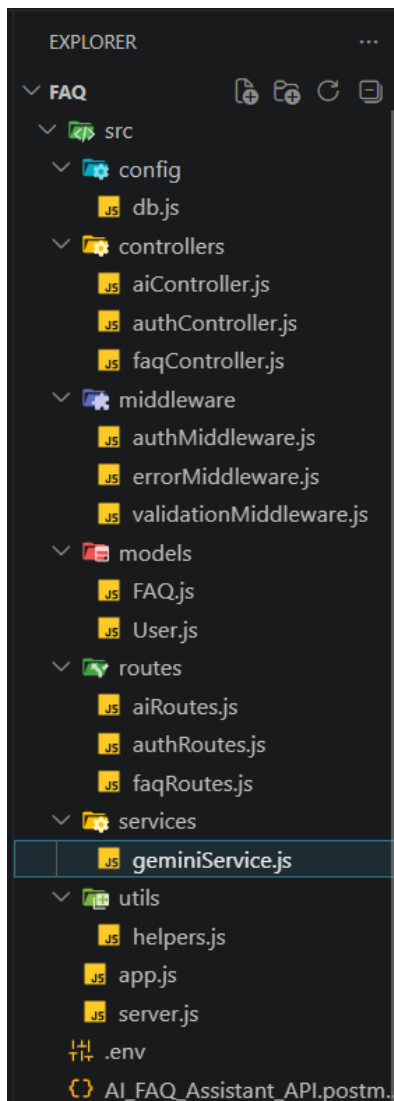
Bash

- **npm init -y**

```
npm install express mongoose dotenv cors jsonwebtoken  
bcrypt @google/genai
```

- File and Folder Structure Matrix

Plaintext.



Epic 3: Backend Development

1. Gateway Server Node (src/server.js)

This main entrance mechanism aggregates environment variables, triggers database listeners, and binds network listeners to active environment ports:

```

c> server.js > ...
1 const app = require('./app');
2 const connectDB = async () => {
3   try {
4     const dbConnect = require('./config/db');
5     await dbConnect();
6   } catch (error) {
7     console.error('Failed to connect to the database', error);
8   }
9 };
10
11 const startServer = async () => {
12   // connect to database
13   await connectDB();
14
15   const PORT = process.env.PORT || 5000;
16
17   const server = app.listen(PORT, () => {
18     console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
19   });
20
21   // Handle unhandled promise rejections
22   process.on('unhandledRejection', (err, promise) => {
23     console.error('Unhandled Rejection: ${err.message}');
24     // Close server & exit process
25     server.close(() => process.exit(1));
26   });
27 };
28
29 startServer();
30

```

1. Gateway Express Setup (src/app.js)

```

src> app.js > ...
1 const express = require('express');
2 const cors = require('cors');
3 const dotenv = require('dotenv');
4 const { errorHandler } = require('./middleware/errorMiddleware');
5
6 // Load environment variables
7 dotenv.config();
8
9 const app = express();
10
11 // Enable CORS
12 app.use(cors());
13
14 // Body parser middlewares
15 app.use(express.json());
16 app.use(express.urlencoded({ extended: false }));
17
18 // Basic status check route
19 app.get('/', (req, res) => {
20   res.json({
21     success: true,
22     message: 'AI FAQ Assistant API is running.',
23     version: '1.0.0',
24   });
25 });
26
27 // Register API Routes
28 app.use('/api/auth', require('./routes/authRoutes'));
29 app.use('/api/faqs', require('./routes/faqRoutes'));
30 app.use('/api/ai', require('./routes/aiRoutes'));
31
32 // Handle Socket.IO polling requests gracefully (if client attempts WebSocket connection)
33 app.use('/socket.io', (req, res) => {
34   res.status(404).json({ error: 'Socket.IO not enabled' });
35 });
36

```

3. Security Access Middleware (src/middleware/authMiddleware.js).

Intercepts incoming target queries, extracts transmission header authorization records, and validates token identity context claims:

```

- > middleware > errorMiddleware.js > ...
1  const errorHandler = (err, req, res, next) => {
2    let statusCode = res.statusCode === 200 ? 500 : res.statusCode;
3    let message = err.message;
4
5    // Respect status code set on the error object (e.g., 429 from Gemini SDK or 400/403/404)
6    if (err.status) {
7      statusCode = err.status;
8    }
9
10   // Mongoose duplicate key error (e.g. duplicate email)
11   if (err.code === 11000) {
12     statusCode = 400;
13     const field = Object.keys(err.keyValue)[0];
14     message = `Duplicate field value entered: ${field}. Please use another value.`;
15   }
16
17   // Mongoose validation error
18   if (err.name === 'ValidationError') {
19     statusCode = 400;
20     message = Object.values(err.errors)
21       .map((val) => val.message)
22       .join(', ');
23   }
24
25   // Mongoose bad ObjectId
26   if (err.name === 'CastError') {
27     statusCode = 404;
28     message = `Resource not found with id of ${err.value}`;
29   }
30
31   // Only log stack trace to the console for unexpected 500 Server Errors
32   if (statusCode === 500) {
33     console.error(err.stack);
34   } else {

```

Epic 4: Database Configuration

Mongoose Schema Definitions

1. Identity Resource Document Schema (src/models/User.js)

Enforces profile field criteria validations and hooks password hashing steps prior to user document persistence:

```

src > models > User.js > ...
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcrypt');
3
4  const userSchema = new mongoose.Schema(
5  {
6    name: {
7      type: String,
8      required: [true, 'Name is required'],
9      trim: true,
10   },
11   email: {
12     type: String,
13     required: [true, 'Email is required'],
14     unique: true,
15     lowercase: true,
16     trim: true,
17     match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email address'],
18   },
19   password: {
20     type: String,
21     required: [true, 'Password is required'],
22     minlength: [6, 'Password must be at least 6 characters long'],
23   },
24 },
25 {
26   timestamps: true,
27 }
28 );
29
30 // Pre-save hook to hash the password before saving
31 userSchema.pre('save', async function () {
32   if (!this.isModified('password')) {
33     return;
34   }
35   const salt = await bcrypt.genSalt(10);
36   this.password = await bcrypt.hash(this.password, salt);
37 });

```

2. FAQ Resource Document Schema (src/models/FAQ.js)

Models core properties and maps a distinct relational reference link back to the source creator document:

```

src > models > FAQ.js > ...
1  const mongoose = require('mongoose');
2
3  const faqSchema = new mongoose.Schema(
4  {
5    question: {
6      type: String,
7      required: [true, 'Question is required'],
8      trim: true,
9    },
10   answer: {
11     type: String,
12     required: [true, 'Answer is required'],
13     trim: true,
14   },
15   category: {
16     type: String,
17     required: [true, 'Category is required'],
18     enum: {
19       values: ['Technology', 'Education', 'Health', 'Banking', 'General'],
20       message: '{VALUE} is not a valid category. Choose from Technology, Education, Health, Banking, ...',
21     },
22     default: 'General',
23   },
24   createdBy: {
25     type: mongoose.Schema.Types.ObjectId,
26     ref: 'User',
27     required: [true, 'Creator user reference is required'],
28   },
29 },
30 {
31   timestamps: true,
32 }
33 );
34
35 const FAQ = mongoose.model('FAQ', faqSchema);
36

```

API Router Interface Endpoints

1. Public Authentication Operations (src/routes/authRoutes.js)

```
.env  geminiService.js  authRoutes.js  AI_FAQ_Assistant_API.postman_collection.json  B...
src > routes > authRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    registerUser,
5    loginUser,
6    getUserProfile,
7  } = require('../controllers/authController');
8  const { protect } = require('../middleware/authMiddleware');
9  const {
10    validateRegister,
11    validateLogin,
12  } = require('../middleware/validationMiddleware');
13
14  // Public routes
15  router.post('/register', validateRegister, registerUser);
16  router.post('/login', validateLogin, loginUser);
17
18  // Protected routes
19  router.get('/profile', protect, getUserProfile);
20
21  module.exports = router;
22
```

2. Protected FAQ Operations (src/routes/faqRoutes.js)

```
.env  geminiService.js  faqRoutes.js  AI_FAQ_Assistant_API.postman_collection.json
> routes > faqRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    createFAQ,
5    getAllFAQs,
6    getFAQById,
7    updateFAQ,
8    deleteFAQ,
9    searchFAQs,
10  } = require('../controllers/faqController');
11  const { protect } = require('../middleware/authMiddleware');
12  const { validateFAQ } = require('../middleware/validationMiddleware');
13
14  // Public routes - GET operations
15  router.get('/search', searchFAQs); // Note: defined before /:id to prevent routing issues
16  router.get('/', getAllFAQs);
17  router.get('/:id', getFAQById);
18
19  // Protected routes - Modification operations
20  router.post('/', protect, validateFAQ, createFAQ);
21  router.put('/:id', protect, updateFAQ);
22  router.delete('/:id', protect, deleteFAQ);
23
24  module.exports = router;
25
```

3. AI Automation Operations (src/routes/aiRoutes.js)

```
rc > routes > aiRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    generateAIAnswer,
5    generateAIFaq,
6  } = require('../controllers/aiController');
7  const { protect } = require('../middleware/authMiddleware');
8  const {
9    validateAIAnswer,
10   validateAIFaq,
11 } = require('../middleware/validationMiddleware');
12
13 // Protected routes to prevent API overuse/abuse
14 router.post('/answer', protect, validateAIAnswer, generateAIAnswer);
15 router.post('/generate-faq', protect, validateAIFaq, generateAIFaq);
16
17 module.exports = router;
18
```

Epic 5: Project Executions

Running the API Application

- Install Postman or ThunderClient (VS Code extension).
- Start the backend development server environment parameters:

Bash

- **npm run dev**
- API Testing Specification Demo

To verify operational structural integrity via execution endpoints, map endpoints using the configurations below:

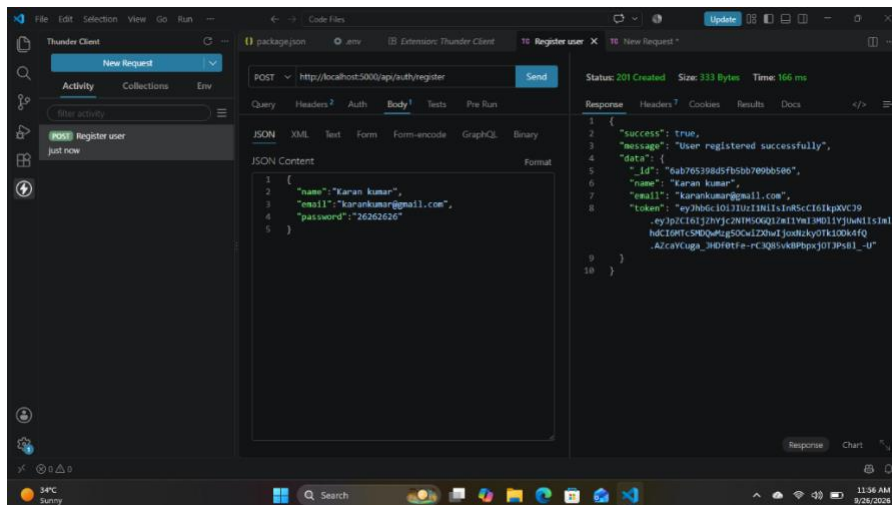
1. Identity Onboarding Path Registration

- Route: POST /api/auth/register
- Access: Public
- Payload Request Body JSON:

JSON

```
{  
  "name": "Priya Sharma",  
  "email": "priya@writeflow.com",  
  "password": "securepassword123"  
}
```

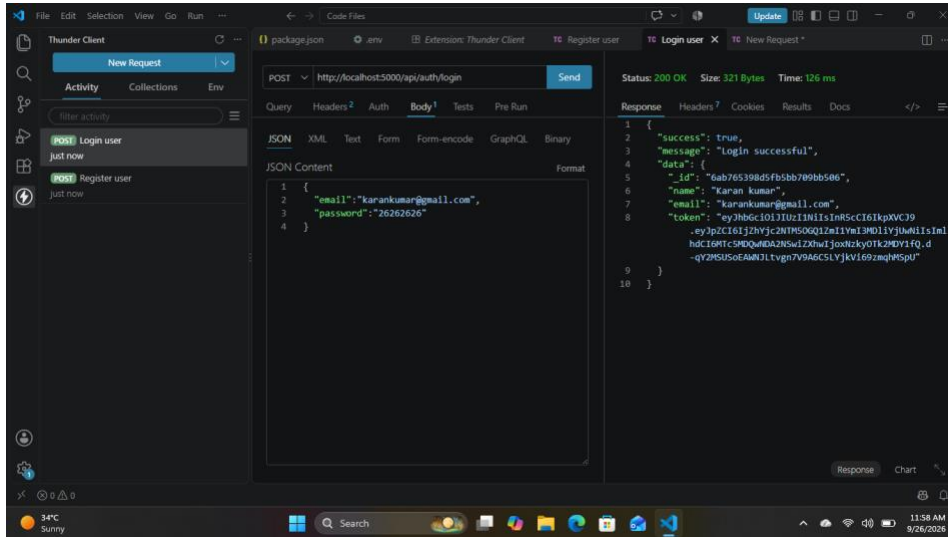
2. Profile Login Verification Quey



- Route: POST /api/auth/login
- Access: Public
- Payload Request Body JSON:

JSON

```
{
  "email": "priya@writeflow.com",
  "password": "securepassword123"
}
```



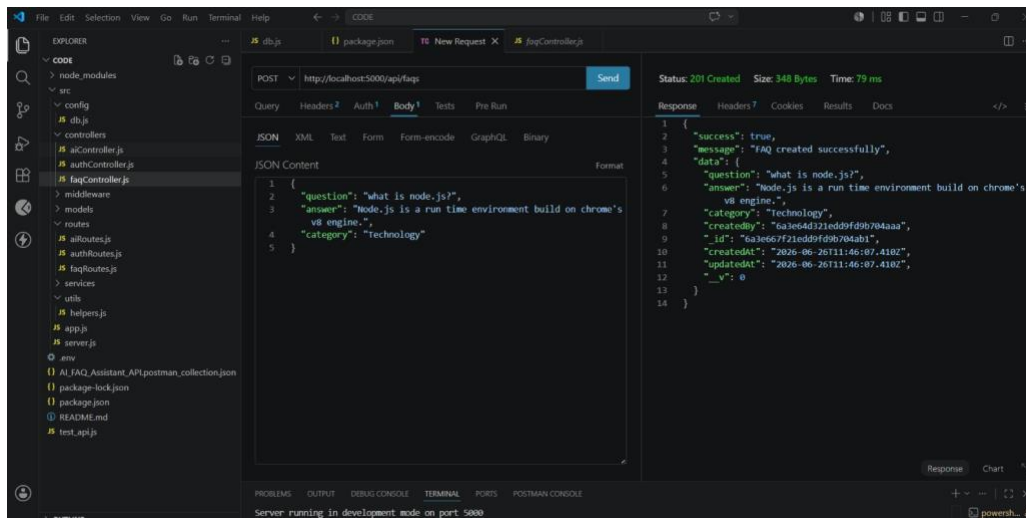
3. Resource Entity Creation (FAQ)

- Route: POST /api/faqs
- Access: Private (Bearer <JWT Token>)
- Payload Request Body JSON:

JSON

```
{
  "question": "How do I configure custom category tags?",
  "answer": "Navigate to settings panel, append keyword tags arrays, and save schema.",
  "category": "Configuration"
```

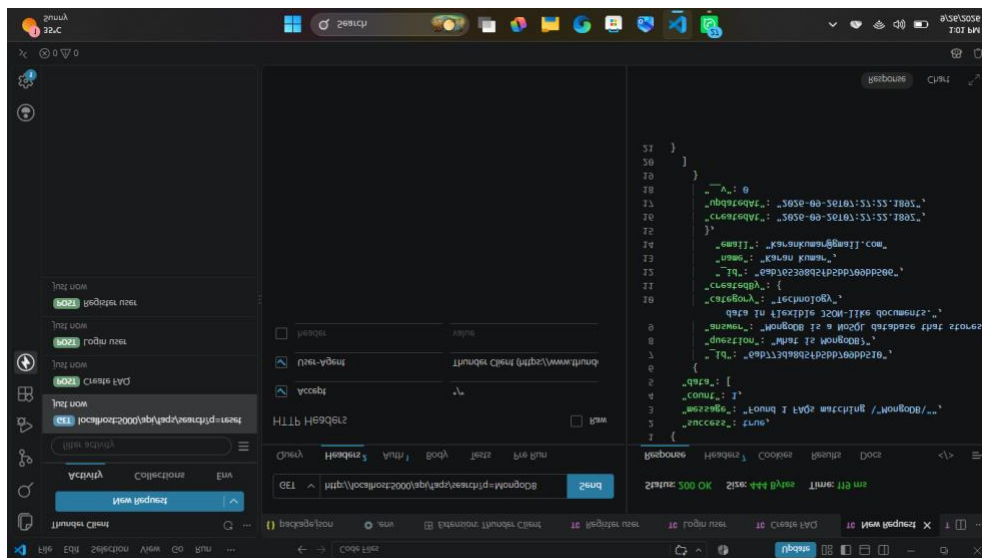
}



. Keyword Resource Entity Fetching (Search)

- Route: GET /api/faqs/search?q=configure

Access: Public



5. AI Intelligent Content Automation (Generate FAQ Pair)

- Route: POST /api/ai/generate-faq
- Access: Private (Bearer <JWT Token>)
- Payload Request Body JSON:

JSON

```
{  
  "topic": "Mongoose schema indexing validation runtime workflow  
optimization"  
}
```

PROJECT DEMO LINK:

https://drive.google.com/file/d/1DXCyMIFqNJNXF02VAgVTRASGt_zQsuiu/view?usp=drivesdk

DRIVE LINK:

https://docs.google.com/document/d/1xFjREp0jYbMU-5Q-tg9oahUiBz_0MMpN/edit?usp=drivesdk&oid=110218752333487222678&rtpof=true&sd=true

